

Introduction to Scientific Computing

ENGR1050 – Lecture 7

Fall 2026

Nat Trask

University of Pennsylvania



End of unit check-in

Basic Concepts



Prep for lab

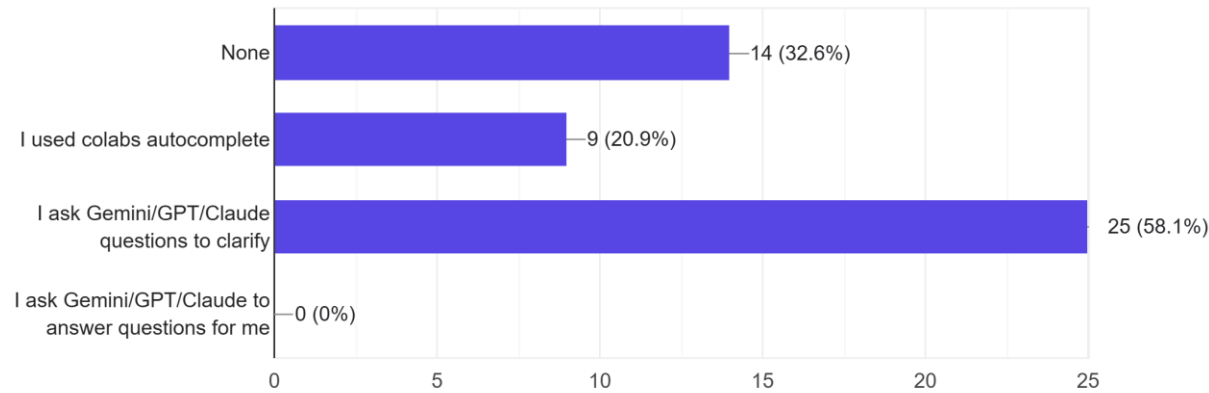
Alternatives to lists

In-class exercise

Practice what we've seen

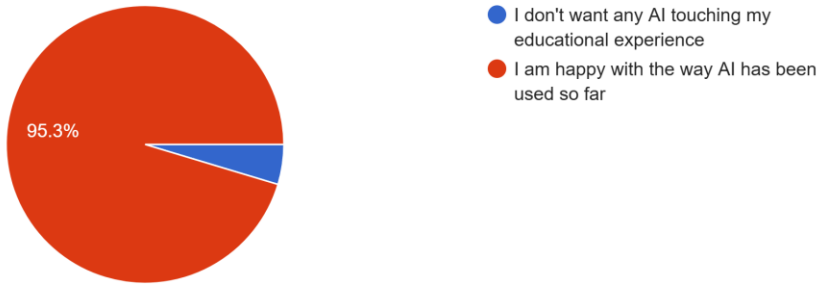
Are you using AI right now in this class?

43 responses



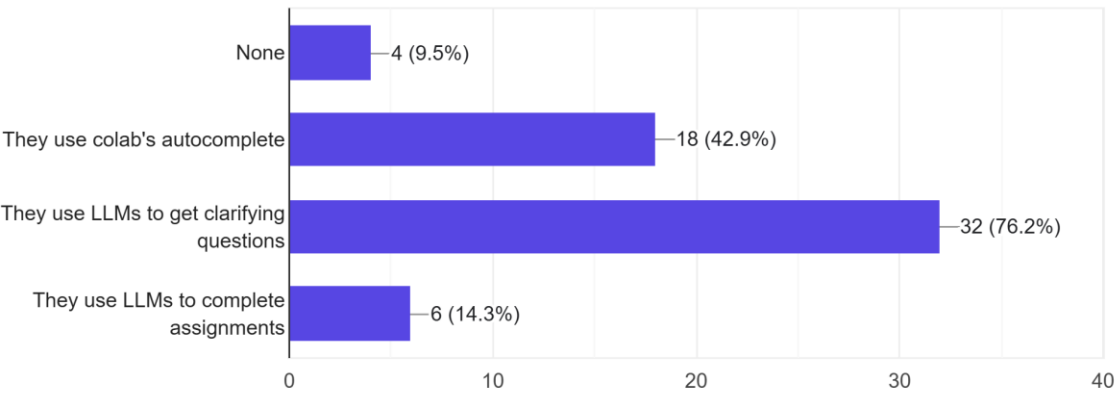
In this class, I (prof trask) use AI to generate additional content. All grading is done by hand. How do you feel about this

43 responses



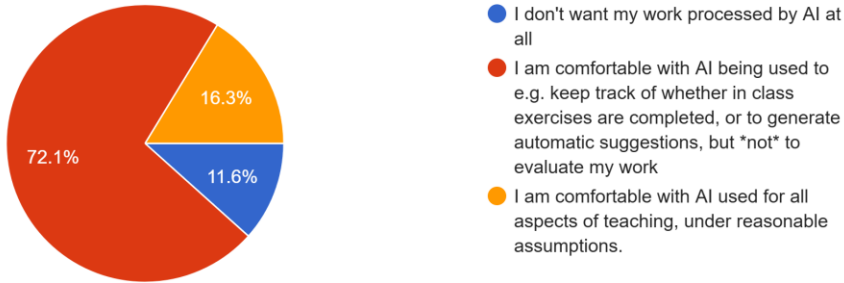
Is your impression that the majority of the class uses AI:

42 responses



There are legal guidelines (called FERPA) which govern how AI may be used to process student data. I (prof trask) am interested in exploring the u.... Before going through this I want to take the pulse:

43 responses



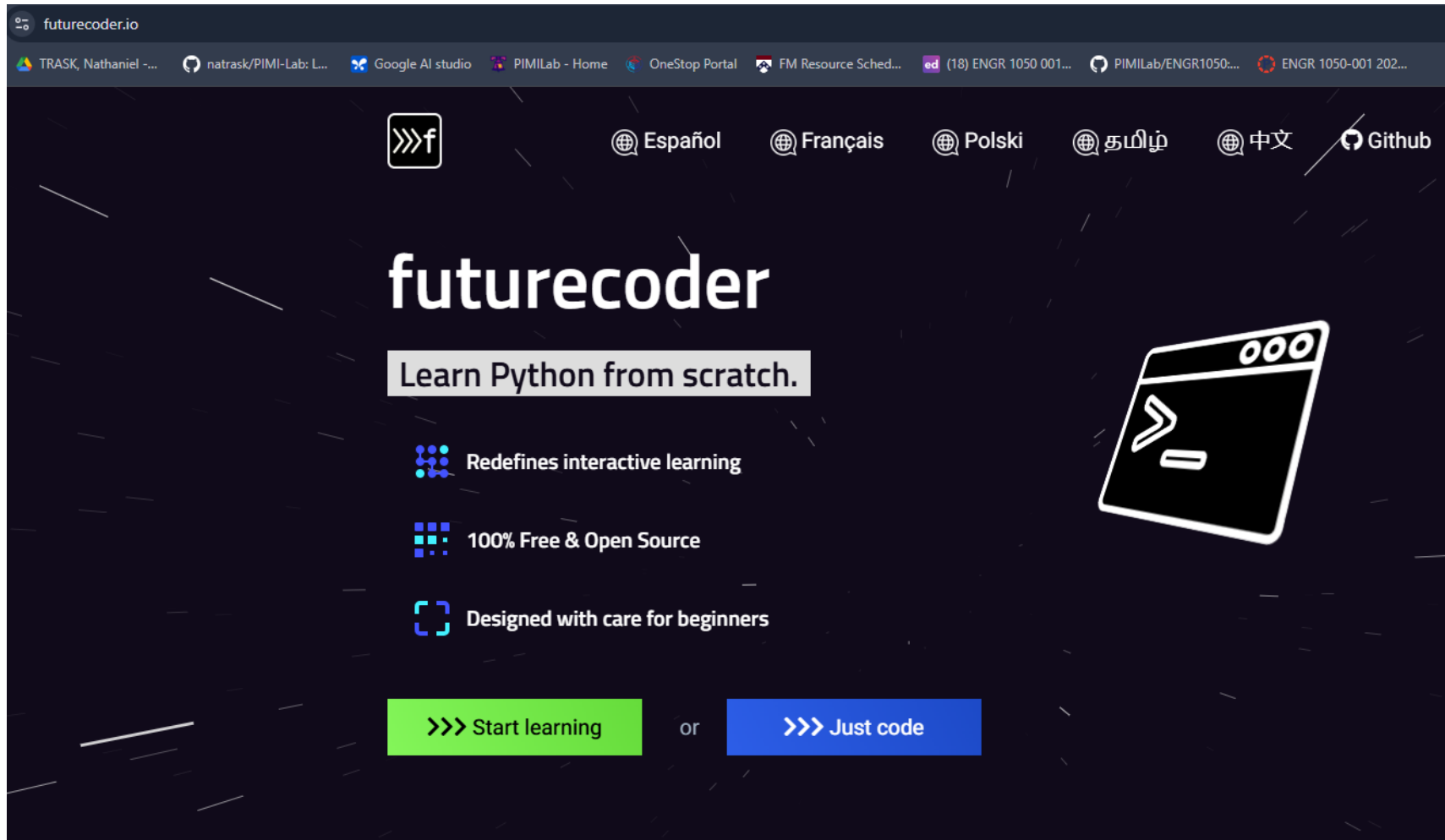
**In-class exercises are hard to push you
to try new things**

Exam will be significantly lighter

**You will practice lists/loops/conditionals/functions/classes for
the rest of the semester and get more comfortable over time**

**In-class exercises today and
Wednesday will help dial in difficulty**

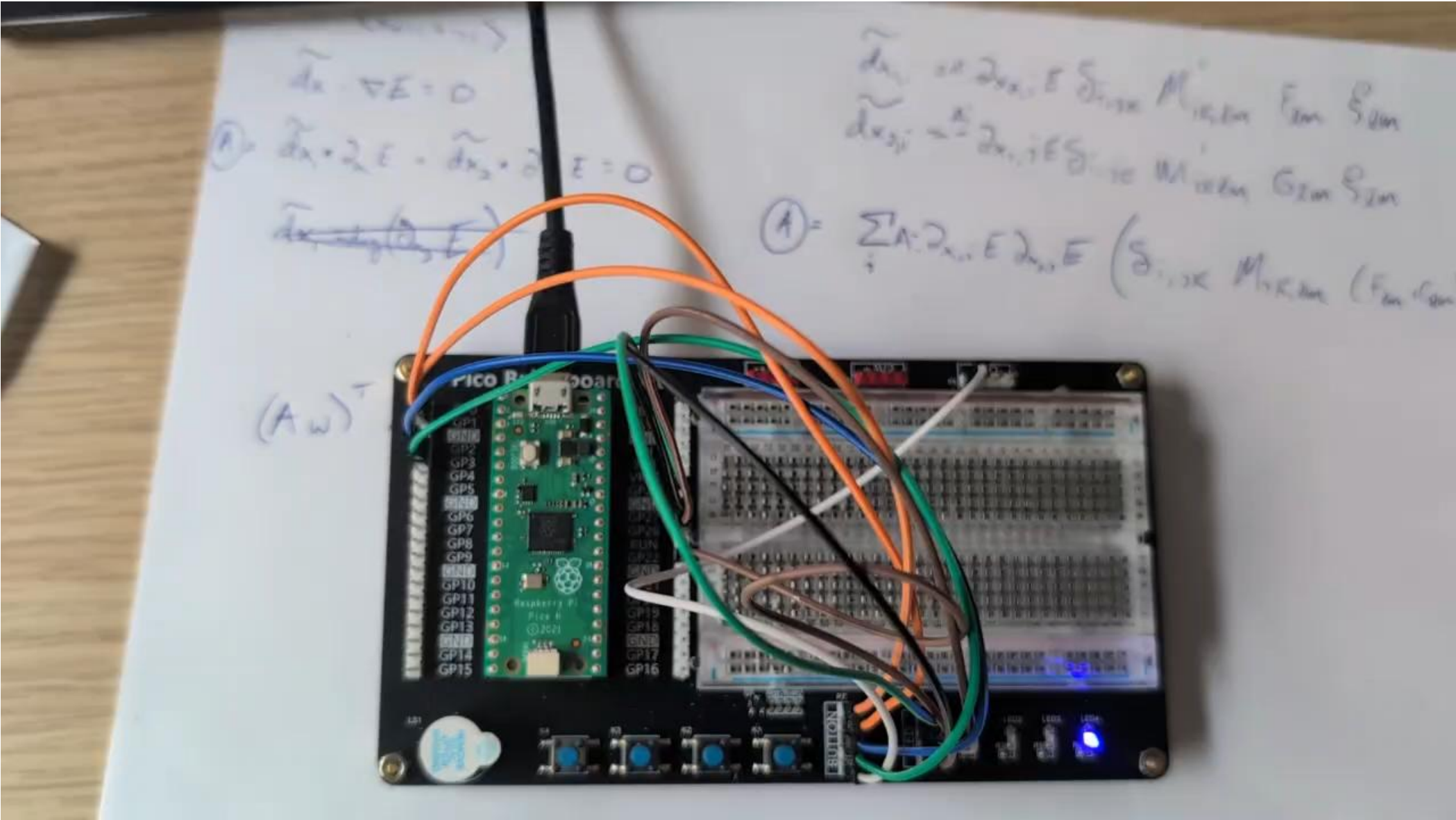
Preparing for exam



futurecoder.io

LearnPython.org

Lab session this Monday



Lab session this Wednesday

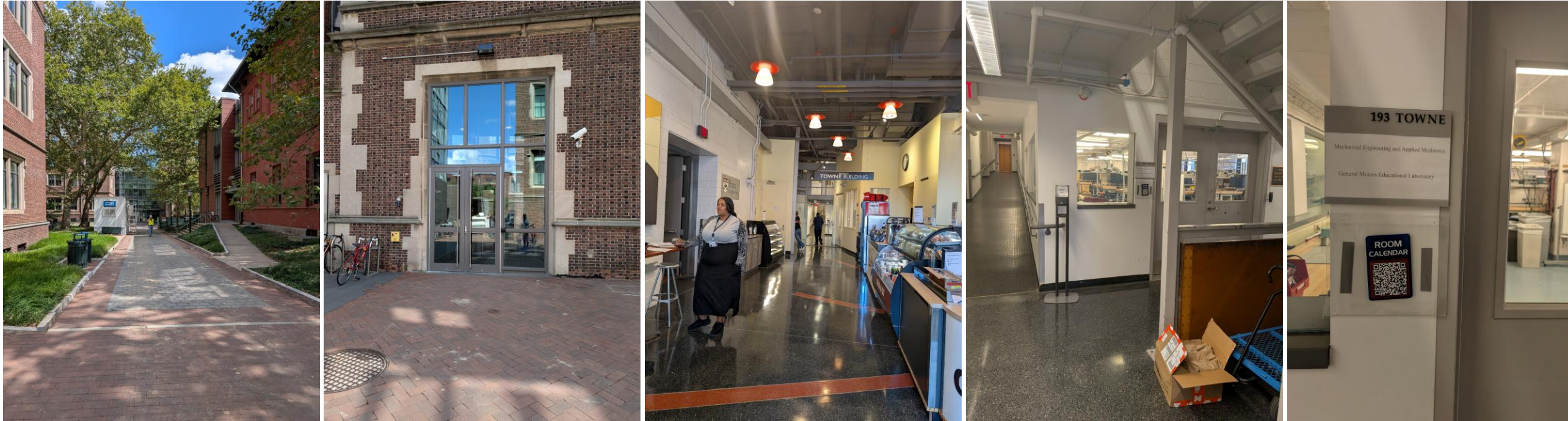
```
1  from machine import Pin
2  import time
3  import random
4  class whackamole:
5      def __init__(self):
6          self.ledlist = [
7              Pin(28, Pin.OUT),
8              Pin(27, Pin.OUT),
9              Pin(26, Pin.OUT),
10             Pin(22, Pin.OUT)
11         ]
12
13         self.buttonlist = [
14             Pin(3, Pin.IN, Pin.PULL_DOWN),
15             Pin(2, Pin.IN, Pin.PULL_DOWN),
16             Pin(1, Pin.IN, Pin.PULL_DOWN),
17             Pin(0, Pin.IN, Pin.PULL_DOWN),
18         ]
19
20         # store the button value at start
21         self.oldbuttonstate = [self.buttonlist[z].value() for z in range(4)]
22
23         #light a random led to start
24         self.lightRandomLED()
25
26     def lightRandomLED(self):
27         for z in range(4):
28             self.ledlist[z].value(0)
29         newled = random.randint(0,3)
30         self.ledlist[newled].value(1)
31
32     def catchButtons(self):
33
34         # check if any of the buttons have flipped their state
35         newbuttonstate = [self.buttonlist[z].value() for z in range(4)]
36
37         for z in range(4):
38             if self.oldbuttonstate[z] == 0 and newbuttonstate[z] == 1:
```

This program will use all of the things we have used so far

In lab I'll provide the code you need so you can focus on the wiring

Today we will work through a similar problem together (without the microcontroller)

Where to find the GM lab



- Will have swipe access
- Arduinos and circuit components will be accessible during class hours
 - Independent group work, submit to canvas when finished
 - For final projects, materials will be accessible

Be respectful of shared space

Classes review

```
class baseClass:
    '''A simple base class with a name and a value that can be incremented.'''
    def __init__(self, name):
        self.name = name
        self.value = 0

    def increment(self):
        self.value += 1

    def display(self):
        return f"Name: {self.name}, Value: {self.value}"
```

```
class derivedClass(baseClass):
    '''A derived class that extends baseClass with an extra attribute and a modified display method.'''
    def __init__(self, name, extra):
        super().__init__(name)
        self.extra = extra

    def display(self):
        return f"Name: {self.name}, Value: {self.value}, Extra: {self.extra}"
```

```
myclass1 = baseClass("className")
myclass2 = derivedClass("derivedName", "extraInfo")
print(myclass1.display())
myclass1.increment()
print(myclass1.display())
print(myclass2.display())
```

More exercises with classes

1. Implement a counter class, which can be used to keep a running tally. It should have the following functions:

- `increment()` - add one to the running tally
- `decrement()` - subtract one from the running tally
- `reset()` - set the tally to 0
- `value()` - return the current value of the tally

```
class CounterClass:
    def __init__(self):
        self.tally = 0

    def increment(self):
        self.tally += 1

    def decrement(self):
        self.tally -= 1

    def reset(self):
        self.tally = 0

    def value(self):
        return self.tally
```

```
mycounter = CounterClass()
print(mycounter.value())
mycounter.increment()
mycounter.increment()
print(mycounter.value())

mycounter2 = CounterClass()
print(mycounter2.value())
mycounter2.increment()
print(mycounter2.value())
```

A function for random number generation

```
import random

def pick_1_to_4():
    """Return a random integer from 1 to 4 inclusive."""
    return random.randint(1, 4)

print(pick_1_to_4())
```

Example whack-a-mole code

```
▶ # Code from exercise will go here.

class Whackamole:
    def __init__(self):
        self.litupslot = random.randint(1,4)

    def whack(self, slotnumber):
        if slotnumber == self.litupslot:
            print("Hit!")
            self.litupslot = random.randint(1,4)

class fileIO:
    def __init__(self):
        self.wheretohit = 0
    def string2number(self,s):
        return int(s)

game = Whackamole()
myfilething = fileIO()
while True:
    print("Type \"exit\" to quit.")
    print(f"The lit up slot is {game.litupslot}")
    user_input = input("Enter a number 1-4 or type quit to quit:")
    if user_input.lower() == "exit":
        break
    else:
        wheretohit = myfilething.string2number(user_input)
        game.whack(wheretohit)
```

